

SOFTWARE ENGINEERING

Architecture Design Assignment · Annexure A



AI-Based Smart Cold Chain Logistics Monitoring & Optimization Platform

A Software Architecture Design Analysis Report



Manisha M (192424130)

1 Introduction & Industry Context

Cold chain logistics is the temperature-controlled supply chain that keeps sensitive goods — vaccines, pharmaceuticals, dairy, seafood, and fresh produce — within a safe environmental range from the point of origin to the point of consumption. Even a short deviation in temperature or humidity can destroy the potency of a vaccine batch or spoil an entire shipment of perishable food, making cold chain integrity a matter of both commercial loss and, in the case of medicine, public safety.

The convergence of Artificial Intelligence, Internet of Things (IoT) sensors, GPS tracking, and cloud computing has transformed what was once a manual, spot-check process into a continuously monitored, predictive system. Sensors stream live telemetry from every truck and container; AI models learn the failure patterns of refrigeration units before they break down; and cloud dashboards give logistics managers a single view of an entire fleet in transit. This report designs the software architecture for exactly such a platform.

2 Problem Statement

Traditional cold chain monitoring relies on periodic manual checks, paper logs, or isolated data-loggers that are only read after a shipment arrives. This gives operators no chance to intervene while a problem is still developing — by the time spoilage is discovered, the loss has already occurred. Equipment failures (compressor faults, power loss, door seal leaks) and transportation delays compound the risk, and the lack of real-time, end-to-end visibility makes it difficult to prove regulatory compliance or pinpoint accountability across a multi-party supply chain.

The platform designed in this report must continuously monitor temperature, humidity, vehicle location, and vibration/door-open events using IoT sensors; use AI to predict refrigeration-equipment failures before they occur; optimise delivery routes against traffic, weather, and cold-retention constraints; generate automated, escalating alerts when conditions drift outside safe thresholds; and present all of this through real-time dashboards for logistics managers, drivers, and compliance officers.

3 System Requirement Analysis

3.1 Key Stakeholders

The system serves six primary stakeholder groups, each with distinct needs:

- Logistics / Fleet Managers — fleet-wide visibility, exception management, KPI reporting.
- Drivers & Field Staff — mobile alerts, route guidance, delivery confirmation.
- Quality & Compliance Officers — audit trails, regulatory reports (GDP/HACCP/FDA-style records).
- Warehouse & Depot Operators — inbound/outbound condition verification.

- Client Organisations (pharma, retail, food producers) — shipment-level tracking and proof of condition.
- System Administrators — platform configuration, user management, device provisioning.

3.2 Functional Requirements

ID	Requirement	Description
FR-1	Real-time telemetry ingestion	Continuously capture temperature, humidity, GPS location, vibration and door-state data from IoT sensors.
FR-2	Threshold-based alerting	Automatically raise alerts when readings breach configured safe ranges, with severity levels and escalation.
FR-3	Predictive maintenance	Use historical + live sensor data to forecast refrigeration-unit failures before they happen.
FR-4	Route optimisation	Recommend delivery routes that minimise transit time and cold-chain risk using live traffic/weather data.
FR-5	Dashboard & visualisation	Provide web and mobile dashboards showing live fleet status, historical trends, and shipment condition.
FR-6	Compliance reporting	Generate exportable audit trails and compliance reports per shipment / regulatory standard.
FR-7	User & role management	Support role-based access for managers, drivers, auditors and administrators.

3.3 Non-Functional (Quality) Requirements

Quality Attribute	Why It Matters for This System
Scalability	Must support a growing number of vehicles, sensors, and concurrent users without redesign.
Reliability	Must operate continuously with minimal downtime; sensor data must not be lost in transit.
Performance	Must process and visualize sensor telemetry with low latency for near real-time alerts.
Availability	Dashboards and alerting must remain available 24/7, including during partial outages.
Security	Sensitive shipment, location, and compliance data must be encrypted and access-controlled.
Maintainability	Individual capabilities should be independently updatable without full-system redeployment.

3.4 Assumptions & Constraints

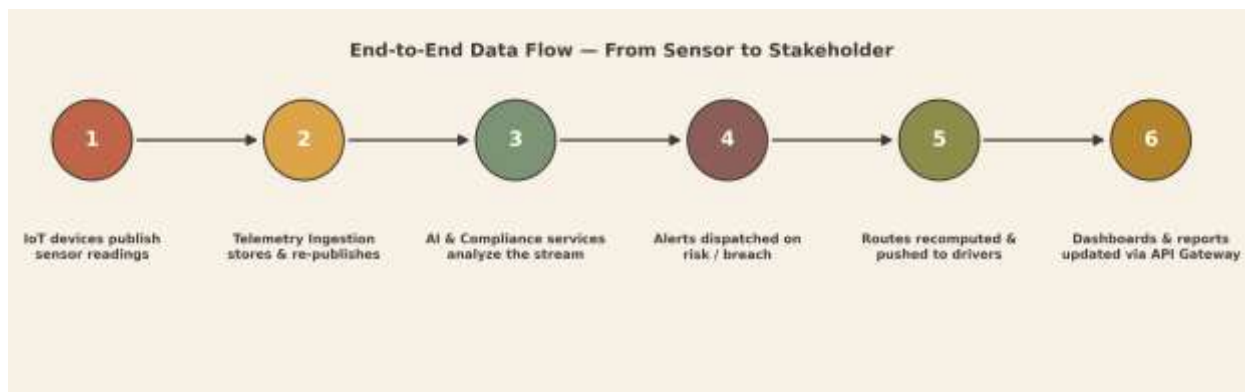
- Vehicles and containers are fitted (or retrofitted) with IoT sensor kits capable of cellular/LoRaWAN connectivity.
- The platform is cloud-hosted (public or hybrid cloud) with regional data residency compliance where required.

- Network connectivity may be intermittent in transit; the design must tolerate temporary offline periods.
- Budget and timeline favour reuse of managed cloud services over fully custom infrastructure.

4 Selection of Software Architecture

4.1 Candidate Architectural Styles Considered

Style	Fit for This Problem
Monolithic	Simple to build initially, but a single deployable unit cannot scale the AI/ML workload independently from ingestion, and any update risks full-system downtime — unsuitable at IoT scale.
Service-Oriented Architecture (SOA)	Improves modularity via an enterprise service bus, but the heavyweight middleware adds latency that conflicts with the need for near real-time alerting.
Layered Architecture	Good for organising concerns (edge, ingestion, processing, data, presentation) but on its own does not address independent scaling or streaming needs.
Event-Driven Architecture	Ideal for high-volume, continuous IoT telemetry and asynchronous alerting, but needs a service-decomposition model layered on top for maintainability.
Microservices	Enables independent scaling and deployment of the AI engine, route optimiser, and alert service, and isolates failures — the strongest fit, provided operational complexity is managed.



- 1) IoT devices on each vehicle/container continuously publish temperature, humidity, GPS, and equipment-status readings to the Event Bus.
- 2) The Telemetry Ingestion Service consumes these events, stores them in the Time-Series Database, and re-publishes normalized events for other consumers.
- 3) The Predictive Maintenance Service and Cold-Chain Compliance Service independently subscribe to the same telemetry stream, each applying their own analysis — failure prediction and threshold compliance, respectively.

- 4) When a risk or breach is detected, an event is published to the Alert & Notification Service, which immediately informs drivers, fleet managers, or quality-control staff.
- 5) The Route Optimization Service consumes GPS and external traffic/weather data to recompute routes, publishing updated routes back through the API Gateway to the driver's mobile app.
- 6) All services expose their data to the Reporting & Analytics Service and the Web Dashboard / Mobile App / Regulator Portal through the API Gateway, which handles authentication, routing, and rate limiting.

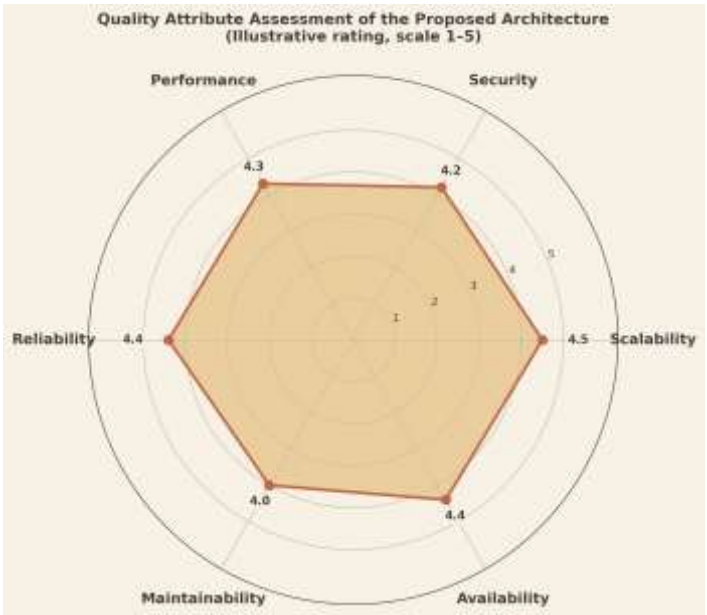
4.2 Selected Architecture: Layered + Event-Driven Microservices (Hybrid)

This report adopts a hybrid architecture that combines a Layered organisation for conceptual clarity (Edge, Connectivity/Ingestion, Cloud Processing & AI, Data, and Application layers) with an Event-Driven Microservices style for the runtime behaviour within and across those layers. Sensor telemetry is treated as a continuous stream of events published to a message broker; independent microservices (stream processing, predictive maintenance, route optimisation, anomaly detection) subscribe to the events they need and scale independently based on load.

This combination was chosen because cold chain monitoring is inherently a streaming problem — thousands of devices emit small, frequent messages rather than making occasional large requests — which event-driven messaging (e.g. MQTT for device ingress and Kafka for internal stream distribution) handles far more efficiently than request/response styles. Microservices decomposition then lets the AI-heavy predictive maintenance and route-optimisation services be scaled, retrained and redeployed independently of the lighter-weight alerting and dashboard services, satisfying the scalability and maintainability requirements identified in Section 3.3 without the operational overhead of a full service mesh at this project's scope.

Quality Attribute Analysis

The proposed hybrid architecture was evaluated against the six quality attributes identified during requirements analysis. The chart below presents an illustrative self-assessment (scale 1–5), followed by a detailed discussion of how the architecture supports each attribute.



Quality Attribute Assessment of the Proposed Architecture

Quality Attribute Analysis

Quality Attribute	Architectural Tactic Used	How It Is Achieved
Scalability	Horizontal scaling + partitioned event streams	Kafka topics are partitioned by vehicle/shipment ID so ingestion and processing scale out linearly as fleet size grows; microservices scale independently via container orchestration.
Reliability / Fault Tolerance	Local buffering, message durability, circuit breakers	Edge gateways buffer during outages; Kafka persists events for replay; services fail independently without cascading.
Availability	Redundancy & managed cloud services	Multi-zone deployment of brokers, databases and microservices with automated failover; target 99.9% uptime for ingestion/alerting.
Security	Defence-in-depth	TLS/mTLS on all links, device certificate authentication, OAuth2/RBAC on APIs, encryption at rest for stored telemetry.
Performance / Latency	Stream processing + caching	In-memory stream processing and a Redis cache in front of the dashboard keep alert delivery within the 5–10 second target.
Interoperability	Open standards	REST/GraphQL APIs and MQTT ingestion allow ERP/WMS and third-party logistics platforms to integrate without custom connectors.
Maintainability	Independent microservice deployment	Each service (AI engine, route optimiser, alerting) is deployed and versioned independently, reducing regression risk on updates.

5 Architecture Diagram & Component Design

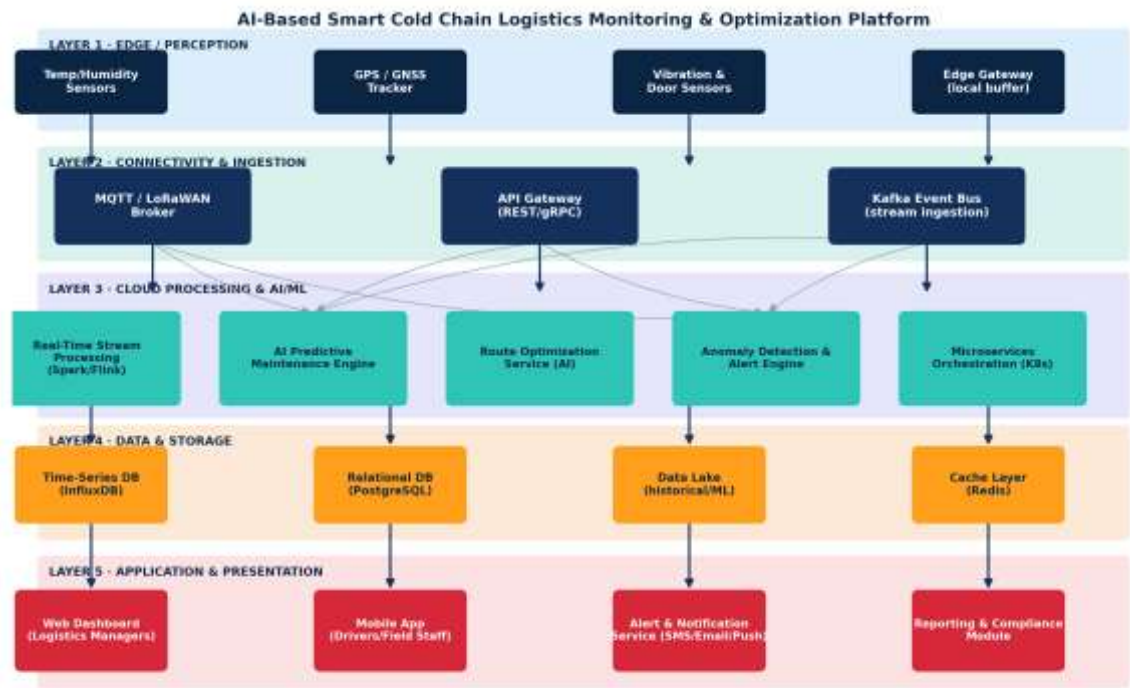


Fig 1. Layered + Event-Driven Microservices Architecture.

Fig. 1 — Layered + Event-Driven Microservices Architecture of the Cold Chain Platform

5.1 Component Responsibilities

Component	Responsibility	Representative Technology
Edge Sensors & Gateway	Capture temperature, humidity, GPS, vibration/door data; buffer locally during connectivity loss.	LoRaWAN / NB-IoT sensor kits, edge micro-controller
MQTT Broker	Lightweight publish/subscribe ingress for low-power, intermittently-connected devices.	Mosquitto / AWS IoT Core
API Gateway	Single entry point for external/partner requests; authentication, rate limiting, routing.	Kong / AWS API Gateway
Kafka Event Bus	Durable, ordered stream distribution across internal microservices.	Apache Kafka
Stream Processing	Windowed aggregation and enrichment of raw telemetry in real time.	Apache Flink / Spark Structured Streaming
AI Predictive Maintenance	Forecasts refrigeration-unit failure probability from sensor + maintenance-history data.	Python (scikit-learn / TensorFlow), model-serving API

Component	Responsibility	Representative Technology
Route Optimisation Service	Computes cold-chain-safe delivery routes using live traffic, weather and vehicle constraints.	Graph/route-optimisation engine + ML ranking
Anomaly Detection & Alerts	Detects threshold breaches and unusual patterns; triggers tiered alerts.	Rules engine + statistical/ML anomaly detection
Data Stores	Time-series DB for telemetry, relational DB for transactional data, data lake for ML training, cache for hot reads.	InfluxDB, PostgreSQL, S3/ADLS data lake, Redis
Application Layer	Web dashboard, mobile app, notification service, and compliance-reporting module.	React/Next.js, native/Flutter mobile, SNS/Twilio

5.2 System Mindmap — Functional Grouping

The mindmap below groups every platform capability into six functional pillars, giving a quick, presentation-friendly view of scope that complements the detailed layered diagram above.



Fig. 2 — Mindmap of Platform Capabilities Grouped by Functional Pillar

6 Component Interaction & Quality Attribute Analysis

6.1 End-to-End Interaction Flow

The following sequence traces a single temperature reading from capture to resolution, illustrating how the components in Section 5 collaborate at runtime:

1. A sensor on a refrigerated container captures temperature and humidity every 30 seconds and publishes the reading to the local Edge Gateway.
2. The Edge Gateway forwards the reading over MQTT to the cloud broker; if connectivity is lost, readings are buffered locally and sent once the link is restored.
3. The MQTT Broker publishes the message onto the Kafka Event Bus, decoupling producers (sensors) from consumers (processing services).
4. The Stream Processing service consumes the event, enriches it with vehicle/shipment metadata, and writes it to the Time-Series Database.
5. In parallel, the Anomaly Detection service evaluates the reading against configured thresholds and recent trend data.
6. If a breach is detected, the Alert Engine publishes an alert event, which the Notification Service turns into an SMS/push/email to the driver and logistics manager within seconds.
7. The AI Predictive Maintenance service continuously re-scores equipment-failure risk using the accumulating time-series history, flagging any unit trending toward failure even before a hard threshold is crossed.
8. The Route Optimisation service, on request or on a significant delay/weather event, recomputes the remaining route to minimise further cold-chain exposure.
9. All events, alerts and actions are simultaneously reflected on the Web Dashboard and Mobile App via the API Gateway, and archived for the Compliance Reporting module.

7.1 Justification of Key Design Decisions

- Event-driven over request/response: cold-chain telemetry is high-frequency and one-directional (device → cloud); a publish/subscribe model avoids the overhead of devices polling or servers polling thousands of devices.
- Microservices over a monolith: the AI predictive-maintenance and route-optimisation workloads are compute-intensive and have very different scaling/release cadences from the lightweight alerting and dashboard services — isolating them prevents one workload from starving another.
- Edge buffering: vehicles routinely pass through connectivity dead-zones; buffering at the gateway guarantees no data is silently lost, which is essential for both safety and regulatory audit trails.

7.2 Trade-offs

Decision	Benefit	Trade-off / Cost Accepted
Microservices decomposition	Independent scaling, fault isolation, faster iteration	Higher operational complexity; requires service discovery, monitoring and orchestration tooling.
Event-driven messaging (Kafka/MQTT)	High-throughput, decoupled, resilient ingestion	Eventual consistency between services; harder to trace a single end-to-end transaction without distributed tracing.
Polyglot persistence	Each store optimised for its workload	More operational surface area (multiple database technologies to maintain and secure).
Edge buffering	No data loss during connectivity gaps	Added firmware/gateway complexity and local storage cost on devices.

7.3 Future Enhancements

- Digital-twin simulation of the fleet to test route and packaging changes before deployment.
- Blockchain-backed provenance ledger for tamper-evident, multi-party compliance records.
- On-device (edge) AI inference for instant spoilage-risk scoring without waiting for a cloud round-trip.
- Computer-vision quality checks at depots to complement sensor data with visual spoilage detection.
- Federated learning across fleets/partners to improve predictive-maintenance accuracy without sharing raw data.
- Carbon-footprint and sustainability analytics layered on top of route-optimisation data.

8 Conclusion

The AI-based Smart Cold Chain Logistics Monitoring and Optimisation Platform designed in this report directly answers the industry problem of limited real-time visibility and reactive spoilage detection. By combining a clearly layered structural view with an event-driven microservices runtime, the architecture supports the scale, reliability and low-latency alerting that temperature-sensitive supply chains demand, while remaining modular enough to evolve — from predictive maintenance today to digital twins, edge AI and sustainability analytics tomorrow. The requirement analysis, architectural comparison, component design and quality-attribute evaluation presented here together form a defensible, professional foundation for implementation.